

PaisaLens

Personalised Financial Behaviour Insight Prototype

Problem statement PS-02 (FinTech) | Track: Prompt to Prototype

Beyond expense tracking: help a person actually understand their own financial behaviour, with guidance that fits their real situation.

Item	Detail
Stack	Next.js 16.3.4 + React 19 + Tailwind CSS 4 + shadcn/ui (Claude theme)
AI layer	Gemini 2.5 Flash via OpenAI-compatible endpoint (swappable model/base URL)
Charts / parsing / schema	Recharts + papaparse + zod
Deploy target	Single Vercel container (web + API routes, no extra server)
Data posture	No database. Browser session only. Account numbers stripped pre-LLM.
Source	doc/PS02_FinTech_Plan.docx + doc/PLAN.md (this repo)

Contents

1. The Problem - what the statement actually asks us to solve
2. Our Solution - PaisaLens in one page
3. Full User Flow - every route, screen, guard rail + 3-minute demo script
4. Architecture in Detail - diagram, client, API contracts, personalisation engine, grounding & safety, data model, file map, LLM config, privacy, UI
5. What 'Good' Looks Like - judging bar + roadmap

1. The Problem

The problem statement does not ask for another expense tracker. It asks for a solution that helps a person **understand their own financial behaviour** and receive **guidance that fits their real situation** - not generic budget rules. Distilled, there are six sub-problems:

- ? **Tracking is not understanding.** A transaction list tells you what you spent, not why it looks that way or whether it aligns with what you care about.
- ? **Context varies wildly.** Students, young professionals and family earners live in very different financial realities - the same rule of thumb cannot work for all three.
- ? **Generic advice feels irrelevant.** 'Save 20% of your income' is easy to say and easy to dismiss. Users tune out anything not grounded in their real data.
- ? **Data is sensitive.** Even a prototype must handle financial data with visible care - no unnecessary storage, no third-party leakage.
- ? **Data is messy.** Real statements have missing categories, cryptic UPI descriptions, duplicates and inconsistent formats.
- ? **Tone matters.** Financial guilt is real. Insights must be clear and non-judgemental, or the user closes the app.

The core differentiator demanded is therefore **not a prettier dashboard but a personalisation layer**: an AI pipeline that reads the user's own spending against their own priorities and speaks to that specific person.

2. Our Solution - PaisaLens

PaisaLens is a lightweight web app with a three-part experience:

- ? **1. Load your money story.** Upload a CSV bank/UPI statement, pick the clearly-labelled synthetic sample month, or type transactions manually. The system cleans and categorises everything, even cryptic descriptions.
- ? **2. Tell us who you are.** One short, respectful screen: life stage (student / professional / family), income pattern (regular / irregular / mixed) and top 2-3 priorities in your own words.
- ? **3. See yourself clearly.** A reflection dashboard: plain-language breakdown, spending rhythm, recurring charges, a priority-alignment meter, and **exactly 3 insight cards** - each expandable to the precise transactions behind it, so nothing feels magical or opaque.

Why it is not 'just another tracker'

- ? **Same numbers, different story.** Change only the life context and the insights visibly change - the personalisation engine ties every observation to a life-stage or stated priority.
- ? **Grounded, never invented.** Every number the AI mentions is computed in code and passed in; every insight cites real transaction IDs the UI can expand.
- ? **Non-judgemental by construction.** Shaming vocabulary is banned in the prompt and re-checked in code; per-slot rule fallback replaces anything that slips.
- ? **Private by design.** Session-only storage, footer privacy note, account numbers stripped before any LLM call, guidance logic open in the repo.

Landed stretch features (beyond MVP)

- ? **WhatIfSimulator** - 'lighter month' playground: skip N deliveries, pause subscriptions, trim shopping X% and see the computed saving. Every rupee traces to real totals.
- ? **LeakDetector** - quiet recurring charges with a yearly lens plus a trip sinking-fund planner.
- ? **ShareCard** - copy/native-share a text summary of the reflection (features + alignment + insights).

3. Full User Flow

Five routes, one shared flow store, guarded transitions:

Route	Screen	What happens
/	Landing	One-line pitch. 'Try sample data' -> /sample. 'Upload my own' -> /upload.
/sample	Sample preview	42-row synthetic month (Aug 2026) in a read-only table with totals. 'Continue with sample' seeds the store -> /context. 'No - upload my own CSV' -> /upload.
/upload	Upload + manual	CSV picker (date,description,amount,type incl. narration/debit variants) via papaparse; manual-entry form; editable category table with delete. sample.csv download link. Continue -> /context (disabled when empty).
/context	Context form	Life stage select, income-pattern select, up-to-3 priority badges. Requires >= 1 priority. Back returns to /sample or /upload based on source. Submit saves context -> /dashboard. Shows pipeline errors with retry.
/dashboard	Pipeline + reflection	On mount: if no transactions, redirect to /upload. Else run categorize -> insights with staged LoadingScreen ('Parsing / Categorising / Reflecting'). Then ReflectionView: inflow/outflow/saved cards, alignment meter, category + rhythm charts, recurring badges, 3 insight cards with txn-trace dialogs, WhatIf + LeakDetector + ShareCard, full txn table. 'Try another context' -> /context (results invalidate + recompute). 'Start over' clears session -> /.

State, guards and refresh behaviour

- ? **FlowProvider** (lib/store.tsx, mounted in layout) holds transactions, context, source (sample/upload), features, insights, model; persisted to sessionStorage keys paisalens:* and rehydrated on load.
- ? **Guards:** /context with zero txns bounces to /sample or /upload; /dashboard with zero txns bounces to /upload; pipeline failure returns to /context with the error shown.

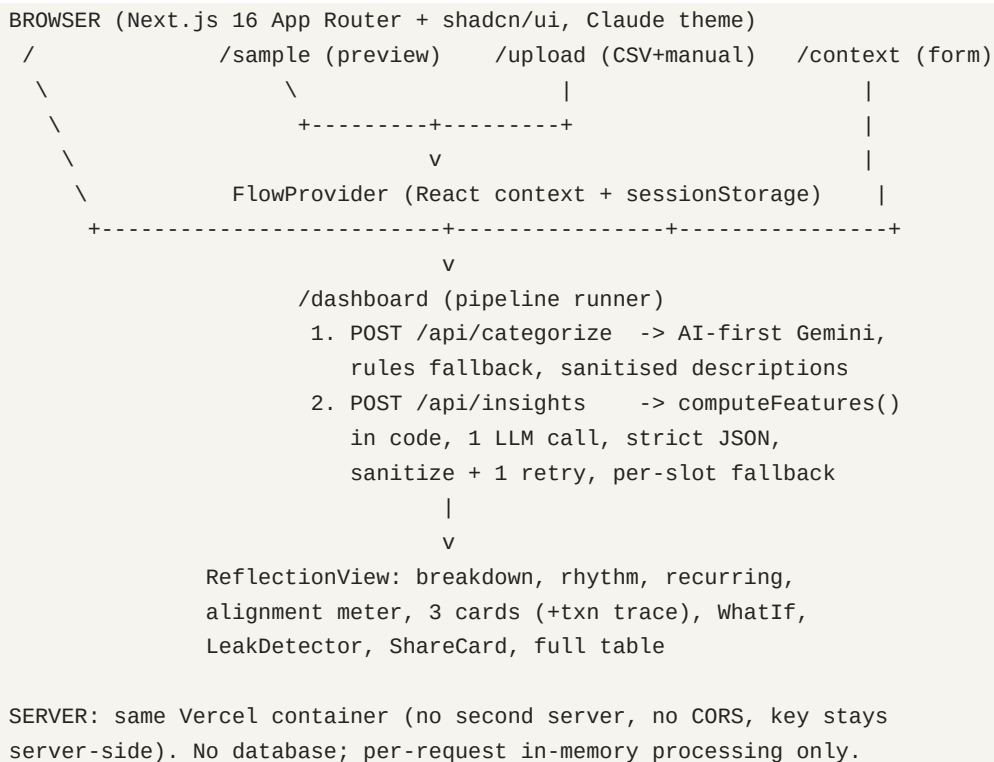
- ? **Refresh on /dashboard** re-runs the pipeline idempotently; the sample path uses the cached response (no LLM cost). User category corrections (confidence = 1) survive re-categorisation.
- ? **Back button** is real browser history - one entry per step, no custom stack.

3-minute demo script

- ? **0:00** Frame it in one line: 'People already track expenses - they still don't understand them.'
- ? **0:15** /sample -> /context as 'college student, irregular pocket money, saving for a laptop'.
- ? **0:45** /dashboard: breakdown + alignment meter. Walk one card: observation, backing transactions, micro-action.
- ? **1:30** The money moment: same transactions, new context ('family earner, regular salary, supporting parents') - insights change while numbers don't.
- ? **2:45** Close on privacy and honesty: no storage, no shaming, no generic advice.

4. Architecture in Detail

4.1 System diagram



4.2 Why Next.js, not plain React

The plan needs server-side API routes to **hide the LLM key**, compute features in code, and deploy as **one Vercel container**. Plain React would force a second server, CORS handling and key-leak risk for zero hackathon benefit.

4.3 API contracts

Endpoint	Input	Output
POST /api/categorize	{ transactions: [{id, description, type, amount}] } (max 200)	{ categories: [{id, category, confidence, source: llm rule default}], llmUsed }
POST /api/insights	{ transactions (categorized), context {lifeStage, incomePattern, priorities[1..3]}, useSampleCache? }	{ insights[3], features, model, llmConfigured }

4.4 Categorisation pipeline (AI-first)

- ? **Step 1 - AI decides.** ALL transaction descriptions are sanitised (card/account numbers -> [card]/[acct], UPI ids -> [upi]) and sent to Gemini 2.5 Flash in ONE call with the 14-category closed list. Accepted only for known ids + valid categories.
- ? **Step 2 - rules cover the gaps.** ~60 keyword patterns over Indian merchants/UPI handles (Swiggy/Zomato, Blinkit/Zepto/DMart, Ola/Uber/Rapido, BESCO/M/Jio/ACT, BookMyShow/PVR, Apollo/1mg/Cultfit, IndiGo/OYO, Netflix/Spotify...) catch anything the model misses.
- ? **Step 3 - safe default.** Anything still unlabelled becomes Income (credits) or Other (debits) at low confidence - categorisation never hard-fails.
- ? **User corrections win.** The pipeline (lib/pipeline.ts) keeps any transaction the user hand-labelled

(confidence = 1). Offline (no key): rules only.

4.5 Personalisation engine (the heart)

- ? **Inputs:** cleaned categorised transactions; declared lifeStage/incomePattern/priorities; code-computed features - category totals, income vs outflow, savings rate, weekday/weekend + first/second-half splits, recurring-charge detection (same-merchant 2+ times), top-5 expenses, income regularity.
- ? **One LLM call** (lib/insights-prompt.ts) with a strict schema: each insight must reference real numbers, tie to a priority/life-stage (showing how the same numbers read for someone different), and include a this-week micro-action (under Rs.500 or free, under 15 min). Shaming words banned.
- ? **Grounding & safety:** numbers computed in code, never by the model; amounts rebuilt from real txn data server-side; tolerant per-insight repair (truncate, backfill txnIds from largest expenses); one retry; per-slot rule-based fallback (lib/fallback-insights.ts); banned-phrase re-check. Token budget 8192 with finish_reason=length truncation detection (Gemini thinking tokens share the budget).
- ? **Demo safety:** useSampleCache path serves a deterministic validated response for the sample month.

4.6 Data model (lib/types.ts)

Type	Fields
Transaction	{ id, date (ISO), description, amount (+ve), type: credit debit, category?, confidence? }
UserContext	{ lifeStage: student professional family, incomePattern: regular irregular mixed, priorities: string[1..3] }
Features	{ income, outflow, savingsRate, categoryTotals[], weekday/weekend, first/secondHalf, subscriptions[], topExpenses[], incomeRegularity, txnCount, periodStart/End }
Insight	{ id, observation, whyItMatters, action, txnIds[], amounts[] }

4.7 File map

```
app/page.tsx, /sample/upload/context/dashboard/page.tsx  (routes)
app/api/categorize/insights/route.ts  (server pipeline)
components/: LandingScreen SamplePreview UploadScreen TxnTable ContextForm
  LoadingScreen ReflectionView InsightCard AlignmentMeter Charts
  WhatIfSimulator LeakDetector ShareCard PrivacyNote + ui/* (shadcn)
lib/: types categories features insights-prompt fallback-insights llm
  pipeline store(sample-data sample-insights.json share utils)
public/sample.csv  (42 synthetic Indian txns)  doc/ (plan + this report)
```

4.8 LLM configuration (swappable)

Env-driven OpenAI-compatible client (lib/llm.ts): **OPENAI_API_KEY**, **LLM_MODEL** (live: gemini-2.5-flash), **LLM_BASE_URL** (live: Google's OpenAI-compatible endpoint). Any OpenAI-compatible provider works - Groq (llama-3.3-70b), OpenRouter, or native OpenAI - with zero code changes. Without a key the app runs fully offline on rules + fallback insights.

4.9 Privacy posture

- ? No database; raw transactions live only in the tab's sessionStorage and per-request server memory.
- ? Account/card numbers and UPI ids are stripped before any LLM call; the LLM mostly sees aggregated features.
- ? Sample data is labelled synthetic; privacy note on upload screen + footer; guidance logic open in the repo.

4.10 UI, theme and dependencies

Strict shadcn/ui on Radix primitives (button, card, dialog, select, table, progress, badge...), Lucide icons, Recharts CategoryBar + RhythmChart, papaparse CSV ingestion, zod validation everywhere. Warm 'Claude' theme tokens (paper background, burnt-orange primary) via Tailwind v4 CSS-first config. Verified: eslint clean, next build clean, all routes + APIs live-tested.

5. What 'Good' Looks Like + Roadmap

Judging bar (from the plan, all met)

- ? **Two contexts, same transactions, visibly different insights** - the single strongest signal.
- ? **Every insight traces to transactions in one click** - no black box.
- ? **Tone of a thoughtful friend**, never a bank app or moralising parent.
- ? **Privacy stated up front and reflected in architecture**, not bolted on.
- ? **End-to-end in under 60 seconds** on the demo URL, upload to insight.

Post-hackathon roadmap

- ? Real bank integration via India's Account Aggregator framework with explicit consent.
- ? Fine-tuned small categorisation model on Indian merchant patterns to lean less on the LLM.
- ? Longitudinal insights (this month vs your own past), goal tracking with gentle nudges.
- ? Multilingual insights (Hindi, Marathi first); optional shared-family mode.